

PRACTICAL WEEK-10

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-10

Task1: Investigate inode structures using commands such as ls -li, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

```
saivihamsh@Varshini-PC:~/practical/week9/task2$ cd ~/practical
saivihamsh@Varshini-PC:~/practical$ mkdir -p week10/task1
saivihamsh@Varshini-PC:~/practical$ cd week10/task1
saivihamsh@Varshini-PC:~/practical/week10/task1$ echo "This is the original file for inode analysis." > original.txt
saivihamsh@Varshini-PC:~/practical/week10/task1$ cat original.txt
This is the original file for inode analysis.
saivihamsh@Varshini-PC:~/practical/week10/task1$ ls -li original.txt
52390 original.txt
saivihamsh@Varshini-PC:~/practical/week10/task1$ stat original.txt
  File: original.txt
  size: 46          Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 52390        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/madhulika)   Gid: ( 1000/madhulika)
Access: 2026-09-26 03:57:23.068735018 +0000
Modify: 2026-09-26 03:57:16.192736556 +0000
Change: 2026-09-26 03:57:16.192736556 +0000
 Birth: 2026-09-26 03:57:16.192736556 +0000
saivihamsh@Varshini-PC:~/practical/week10/task1$ ln original.txt hardlink.txt
saivihamsh@Varshini-PC:~/practical/week10/task1$ ls -li original.txt hardlink.txt
52390 hardlink.txt 52390 original.txt
saivihamsh@Varshini-PC:~/practical/week10/task1$ stat original.txt
  File: original.txt
  size: 46          Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 52390        Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/madhulika)   Gid: ( 1000/madhulika)
Access: 2026-09-26 03:57:23.068735018 +0000
Modify: 2026-09-26 03:57:16.192736556 +0000
Change: 2026-09-26 03:57:57.380734010 +0000
 Birth: 2026-09-26 03:57:16.192736556 +0000
saivihamsh@Varshini-PC:~/practical/week10/task1$ ln -s original.txt symlink.txt
saivihamsh@Varshini-PC:~/practical/week10/task1$ ls -li original.txt hardlink.txt symlink.txt
52390 hardlink.txt 52390 original.txt 52391 symlink.txt
```

```

saiviamsh@Varshini-PC:~/practical/week10/task1$ stat symlink.txt
  File: 'symlink.txt' -> 'original.txt'
 size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 52391         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/madhulika)   Gid: ( 1000/madhulika)
Access: 2026-09-26 03:58:20.152729210 +0000
Modify: 2026-09-26 03:58:15.260730270 +0000
Change: 2026-09-26 03:58:15.260730270 +0000
 Birth: 2026-09-26 03:58:15.260730270 +0000
saiviamsh@Varshini-PC:~/practical/week10/task1$ readlink symlink.txt
original.txt
saiviamsh@Varshini-PC:~/practical/week10/task1$ find . -inum "$(ls -li or
iginal.txt | awk '{print $1}')"
./original.txt
./hardlink.txt
saiviamsh@Varshini-PC:~/practical/week10/task1$ ls -li
total 8
52390 -rw-r--r--  2 madhulika madhulika 46 Sep 26 03:57 hardlink.txt
52390 -rw-r--r--  2 madhulika madhulika 46 Sep 26 03:57 original.txt
52391 lrwxrwxrwx  1 madhulika madhulika 12 Sep 26 03:58 symlink.txt -> origin
al.txt
saiviamsh@Varshini-PC:~/practical/week10/task1$

```

Observation: The inode structure of files was successfully investigated using `ls -li`, `stat`, and `find`. A hard link was created for the original file, and both files were found to share the same inode number with a link count of 2. A symbolic link was also created, which had a separate inode and pointed to the original file. The final `ls -li` output clearly showed the inode allocation and link relationships.

Task2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```

saivihamsh@Varshini-PC:~/practical/week10/task1$ cd ~/practical/week10
saivihamsh@Varshini-PC:~/practical/week10$ mkdir -p task2
saivihamsh@Varshini-PC:~/practical/week10$ cd task2
saivihamsh@Varshini-PC:~/practical/week10/task2$ nano mmap_compare.c
saivihamsh@Varshini-PC:~/practical/week10/task2$ gcc mmap_compare.c -o T2
saivihamsh@Varshini-PC:~/practical/week10/task2$ ./T2
=== Traditional read/write ===
Data: Week-10 Practical Task 2: Memory mapped file I/O demonstration.
◆◆ Time: 0.000018 seconds

=== Memory-mapped I/O ===
Data: Week-10 Practical Task 2: Memory mapped file I/O demonstration.
Time: 0.010074 seconds

File size: 4096 bytes

Memory-mapped I/O demonstration completed successfully.
saivihamsh@Varshini-PC:~/practical/week10/task2$ cat mmap_file.txt
Week-10 Practical Task 2: Memory mapped file I/O demonstration.
saivihamsh@Varshini-PC:~/practical/week10/task2$ ls -l mmap_file.txt
-rw-r--r-- 1 madhulika madhulika 4096 Sep 26 04:04 mmap_file.txt
saivihamsh@Varshini-PC:~/practical/week10/task2$ ./T2
=== Traditional read/write ===
Data: Week-10 Practical Task 2: Memory mapped file I/O demonstration.
◆◆ Time: 0.000014 seconds

=== Memory-mapped I/O ===
Data: Week-10 Practical Task 2: Memory mapped file I/O demonstration.
Time: 0.005735 seconds

File size: 4096 bytes

Memory-mapped I/O demonstration completed successfully.
saivihamsh@Varshini-PC:~/practical/week10/task2$

```

```

GNU nano 8.7.1      mmap_compare.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

#define FILE_SIZE 4096

double get_time(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(void)
{
    int fd;
    char buffer[FILE_SIZE];
    struct stat st;
    double start, end;

    const char *data =
        "Week-10 Practical Task 2: Memory mapped file I/O demonstration.\n";

    /* Create initial file */
    fd = open("mmap_file.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (ftruncate(fd, FILE_SIZE) == -1) {
        perror("ftruncate");
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```
GNU nano 8.7.1 mmap_compare.c *
    perror("read");
    close(fd);
    return 1;
}

end = get_time();

printf("=== Traditional read/write ===\n");
printf("Data: %s", buffer);
printf("Time: %.6f seconds\n\n", end - start);

/* mmap */
start = get_time();

char *mapped = mmap(NULL, FILE_SIZE,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);

if (mapped == MAP_FAILED) {
    perror("mmap");
    close(fd);
    return 1;
}

strcpy(mapped, data);

if (msync(mapped, FILE_SIZE, MS_SYNC) == -1) {
    perror("msync");
    munmap(mapped, FILE_SIZE);
    close(fd);
    return 1;
}

printf("=== Memory-mapped I/O ===\n");
printf("Data: %s", mapped);

munmap(mapped, FILE_SIZE);

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Observation: The program successfully demonstrated file reading and writing using both traditional read()/write() operations and memory-mapped I/O using mmap(). The same data was successfully written and read using both methods, and their execution times were measured for comparison. The file size was also verified after the operations.